The fast multipole method.

1. Description of the algorithm with some discussion.
2. Implementation in Python and quick complexity experiments.

---

1. On the fast multipole method. A fast multipole method can reduce the computational complexity of the Coulomb interaction

$$\frac{1}{N}\sum_{i\neq j}\frac{1}{\lambda_i-\lambda_j}$$

from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ (or $\mathcal{O}(N\log N)$) by, informally, collecting the influence of 'distant' particles together and only explicitly calculating the repulsion for 'near' particles.

This technique was first described by Greengard and Rokhlin [1] and has since had applications in many problems.

High level conceptual understanding. The main idea behind the algorithm is to partition the repulsion influence between near and far neighbours. Near neighbours have their repulsion calculated exactly, whereas distant neighbours make use of multipole and local expansions to efficiently approximate their influence.

Notions of near and far are determined by a binary tree structure.

More technically. Let $(\lambda_i)_{1\leq i\leq N}$ be the (sorted) particle system (i.e. $\lambda\in\mathcal{W}_N$). Recursively bisect the particle support into a binary tree until each leaf contains at most s particles. Denote the centre of a box B as c_B.

Consider some λ_j in any box B. Then, for a distant target x, one has

$$\sum_{\lambda_j\in B}\frac{1}{x-\lambda_j}=\sum_{\lambda_j\in B}\frac{1}{(x-c)-(\lambda_j-c)}=\sum_{\lambda_j\in B}\frac{1}{x-c}\frac{1}{1-\frac{\lambda_j-c}{x-c}},$$

representing the combined influence of all particles in B on the target x. By defining $r:=(\lambda_j-c)/(x-c)$, and noting $|r|<1$ by the distant nature of x,

$$\sum_{\lambda_j\in B}\frac{1}{x-\lambda_j}=\sum_{\lambda_j\in B}\frac{1}{x-c}\frac{1}{1-r}=\sum_{\lambda_j\in B}\frac{1}{x-c}\sum_{k=0}^{\infty}r^k.$$

A swap of the summations and re-substitution of r yields

$$\sum_{\lambda_j\in B}\frac{1}{x-\lambda_j}=\sum_{k=0}^{\infty}\frac{1}{(x-c)^{k+1}}\sum_{\lambda_j\in B}(\lambda_j-c)^k=:\sum_{k=0}^{\infty}\frac{M_k}{(x-c)^{(k+1)}},$$

where M_k is sometimes referred to as the multipole moment. Here the infinite sum is truncated to the first p terms.

This form gives the multipole expansion for any box, but the moments are only explicitly calculated for the leaves. If M_k^C are the moments for some child node, and M_k^P the moments for the parent node, notice one can recentre

$$(\lambda_j-c_P)^k=((\lambda_j-c_C)+(c_C-c_P))^k=:((\lambda_j-c_C)+\delta)^k$$

with $\delta = c_C - c_P$. By standard binomial expansion

$$(\lambda_j - c_P)^k = \sum_{n=0}^{k} \binom{k}{n} (\lambda_j - c_C)^n \delta^{k-n}$$

and therefore the moments of a parent node can be expressed entirely in terms of the moments of its children,

$$\begin{aligned} M_k^P &= \sum_{\lambda_j \in P} (\lambda_j - c_P)^k = \sum_{n=0}^{k} \binom{k}{n} (c_C - c_P)^{k-n} \sum_{\lambda_j \in P} (\lambda_j - c_C)^n \\ &= \sum_{n=0}^{k} \binom{k}{n} M_n^{C_L} (c_{C_L} - c_P)^{k-n} + \sum_{n=0}^{k} \binom{k}{n} M_n^{C_R} (c_{C_R} - c_P)^{k-n}, \end{aligned}$$

where C_L and C_R are the two children boxes of P, i.e. $P = C_L \cup C_R$. This completes the upward pass.

Now we form a local expansion for each box, working top down. Let $\mathcal{I}(B)$ denote the interaction list of a box: the boxes on the same level as B, not adjacent to B, but whose parents are adjacent to B's parent (so they would not have been considered at the previous level).

Consider the p-multipole expansion of each $S \in \mathcal{I}(B)$ and define $R := (c_B - c_S)$ in order to recentre the expansion in B,

$$\sum_{k=0}^{p} \frac{M_k^S}{(x - c_S)^{k+1}} = \sum_{k=0}^{p} \frac{M_k^S}{((x - c_B) + (c_B - c_S))^{k+1}} = \sum_{k=0}^{p} \frac{M_k^S}{((x - c_B) + R)^{k+1}}.$$

Take the standard binomial expansion

$$(1 + z)^{-m} = \sum_{n=0}^{\infty} (-1)^n \binom{m + n - 1}{n} z^n$$

of the denominator so that

$$((x - c_B) + R)^{-(k+1)} = R^{-(k+1)} \sum_{n=0}^{\infty} (-1)^n \binom{n + k}{n} \left(\frac{x - c_B}{R} \right)^n.$$

Then

$$\sum_{k=0}^{p} \frac{M_k^S}{(x - c_S)^{k+1}} = \sum_{k=0}^{p} M_k^S R^{-(k+1)} \sum_{n=0}^{\infty} (-1)^n \binom{n + k}{n} \left(\frac{x - c_B}{R} \right)^n.$$

Recall that the goal is to form a local expansion for each box, collecting the influence of all multipole expansions on a target x around the centre c_B. Local expansions take the form

$$L^B(x) = \sum_{n=0}^{p} L_n^B (x - c_B)^n$$

for coefficients L_k^B, where again the series has been truncated to order p. This can be achieved by truncating the n-series and rearranging so that

$$\begin{aligned}\sum_{k=0}^{p} \frac{M_k^S}{(x-c_S)^{k+1}} &= \sum_{n=0}^{p} (x-c_B)^n (-1)^n \sum_{k=0}^{p} \binom{n+k}{n} \frac{M_k^S}{R^{n+k+1}} \\ &=: \sum_{n=0}^{p} (x-c_B)^n L_n^B.\end{aligned}$$

This only considers each individual box $S \in \mathcal{I}(B)$, with the full local expansion accumulating information over all $S \in \mathcal{I}(B)$ by taking

$$L_n^B = \sum_{S \in \mathcal{I}(B)} (-1)^n \sum_{k=0}^{p} \binom{n+k}{n} \frac{M_k^S}{R_S^{n+k+1}}, \qquad R_S = c_B - c_S.$$

Once a local expansion for a box B has been computed, it is passed down to its children C by again reformulating the polynomial $(x-c_B)^n L_n^B$ around c_C. Define $d = c_C - c_B$ and consider

$$(x-c_B)^n = ((x-c_C)+d)^n = \sum_{m=0}^{n} \binom{n}{m} (x-c_C)^m d^{n-m}$$

so that (being careful with the indices here)

$$\begin{aligned}L^B(x) &= \sum_{n=0}^{p} \sum_{m=0}^{n} \binom{n}{m} (x-c_C)^m d^{n-m} L_n^B \\ &= \sum_{m=0}^{p} (x-c_C)^m \sum_{n=m}^{p} \binom{n}{m} L_n^B d^{n-m} =: \sum_{m=0}^{p} (x-c_C)^m L_m^C.\end{aligned}$$

The child's local expansion continues to accumulate the contribution of all boxes in its interaction list, with this information being passed all the way down to the leaves.

Finally, the Coulomb interaction for each particle λ_i is approximated as the sum of all far field interactions from the local expansion, and direct interactions from explicit calculation. That is, for a leaf L and its adjacent leaves $\mathcal{A}(L)$

$$\frac{1}{N} \sum_{i \neq j} \frac{1}{\lambda_i - \lambda_j} \approx \sum_{k=0}^{p} L_k^L (\lambda_i - c_L)^k + \frac{1}{N} \sum_{\substack{j \in (L \cup \mathcal{A}(L)) \\ i \neq j}} \frac{1}{\lambda_i - \lambda_j}.$$

One can draw a correspondence between these steps and the more general case in [1] as follows: multipole expansions for the leaves are formed in step one, and then propagated up the tree through their parents in step two. Step three handles the local expansion for each box, with step four passing these down through the tree. Steps five, six, and seven calculate the local expansions and direct contributions for each particle.

These authors also provide formal error analysis associated with the infinite series truncation, showing that the error decays geometrically in p and independently of N. For a fixed precision the asymptotic convergence of FMM is $\mathcal{O}(N)$, but the inherent constant is large. Implementing FMM with only far field expansion (no local expansions) yields an algorithm of $\mathcal{O}(N \log(N))$ complexity; see the discussion in [2].

2. Implementation in Python. See `fmm.py`. The upward pass is described through M2M (multipole to multipole), with local expansions being formed in M2L (multipole to local) and then being passed down to children through L2L (local to local).

A small and reasonably naive experiment has been run, investigating the clock time of calculating an N-coulomb interaction both directly and with FMM. Figure 1 demonstrates the clock time with N for various levels of precision p, with Figure 3 showing the average and maximal errors across a small number of trials (here just three).

The large constant in the asymptotic $\mathcal{O}(N)$ convergence of FMM is apparent. For experiments with small to moderate dimension ($N \leq 5000$, say) using the naive Coulomb interaction is faster - and there's no error involved! However, if the dimension grows - particularly to $N > 10^4$ - using FMM is imperative.

Other algorithms that are variants of (or otherwise related to) FMM were also considered: namely the Barnes-Hut algorithm [3] of asymptotic $\mathcal{O}(N \log(N))$ complexity. We found that FMM outperformed this variant for all moderate N, although its error magnitude was significantly smaller. As the Coulomb interaction is just one component of a noisy system anyway, errors of $\approx 10^{-3}$ feel more than tolerable for improved performance.

Figure 2 provides visual comparison of the methods when compiled into machine code with Numba. See also the discussion in [4].

References

[1] Leslie Greengard and Vladimir Rokhlin. "A fast algorithm for particle simulations". In: *Journal of Computational Physics* 73.2 (1987), pp. 325–348.

[2] Norman Yarvin and Vladimir Rokhlin. "A generalized one-dimensional fast multipole method with application to filtering of spherical harmonics". In: *Journal of Computational Physics* 147.2 (1998), pp. 594–609.

[3] Josh Barnes and Piet Hut. "A hierarchical $\mathcal{O}(N \log N)$ force-calculation algorithm". In: *Nature* 324.6096 (1986), pp. 446–449.

[4] Guy Blelloch and Girija Narlikar. "A practical comparison of n-body algorithms". In: *Parallel Algorithms* 30 (1997), pp. 81–96.

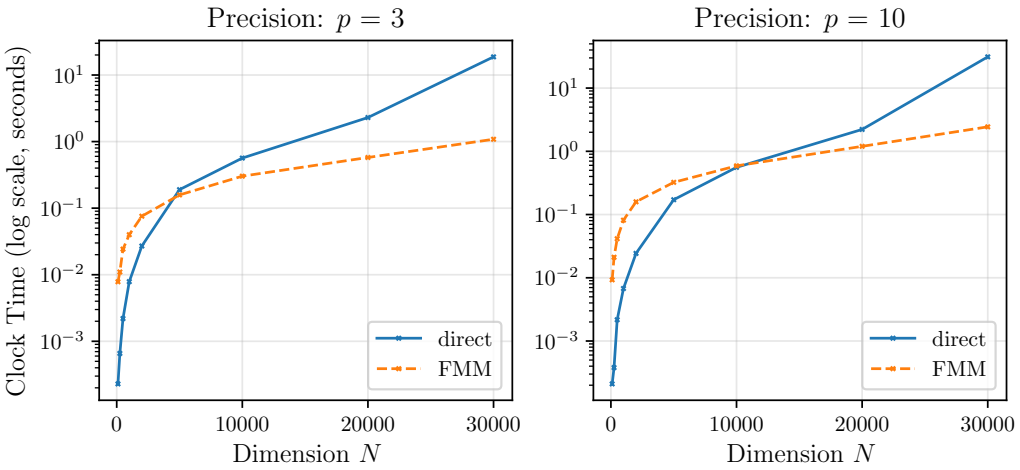

Figure 1: Clock time (elapsed time on hardware) of the direct and FMM calculation of the Coulomb interaction.

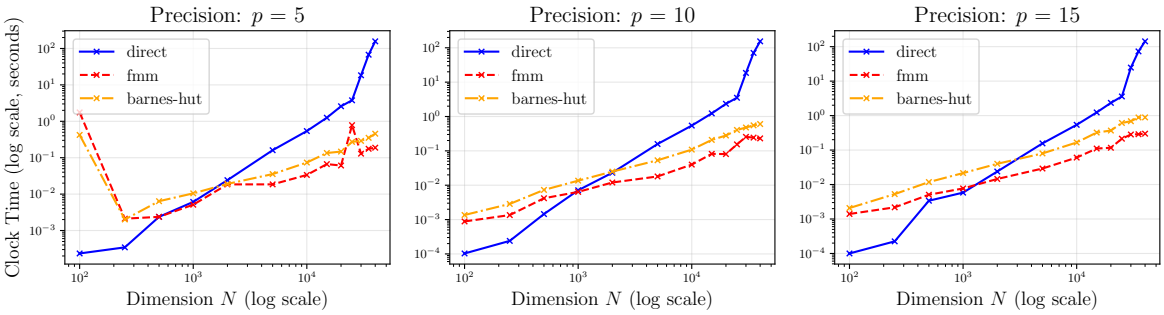

Figure 2: Clock time for the Numba-compiled FMM and Barnes-Hut algorithms.

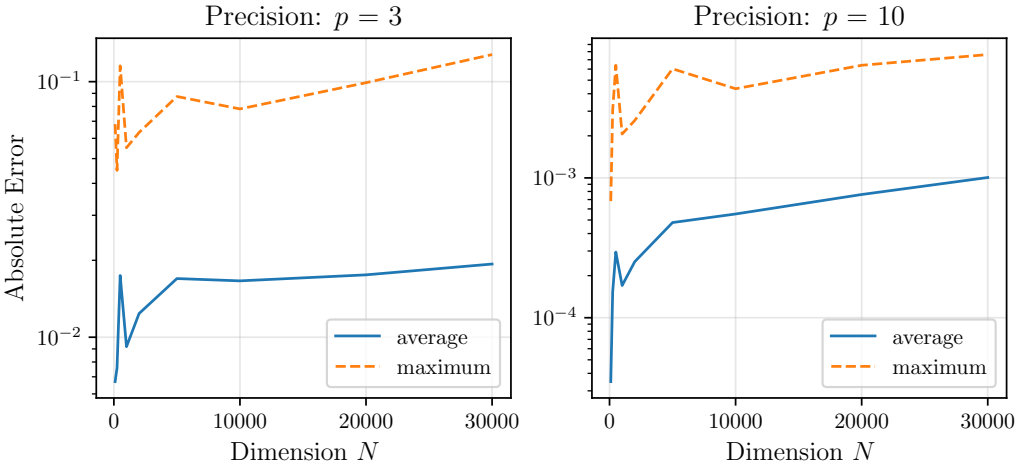

Figure 3: Average and maximal errors of the FMM Coulomb approximation for different levels of precision p.